

**Also known as:** Code Byte, coding assessment, coding round, DSA practice, the 5-phase test

**Audience:** candidate

**Last verified:** 2026-09-18

# Code Byte — the five-phase coding assessment

## 1. What this is

Code Byte does not just check whether your code passes the tests. It walks you through the way a good engineer actually solves a problem — understand it, plan it, write it, improve it, explain it — and scores each of those separately. Two candidates whose code both passes can get very different Code Byte scores, because one understood the problem and one guessed until the tests went green.

There is also a simpler **daily coding practice** arena where you write a solution and run it against test cases, without the five phases.

## 2. Who can use it

Every candidate, on both plans.

## 3. Names and terms

| A user might say                   | Meaning                                                        |
|------------------------------------|----------------------------------------------------------------|
| Code Byte, coding assessment       | the five-phase assessment                                      |
| coding practice, daily coding, DSA | the simpler run-the-tests arena                                |
| phase                              | one of the five stages                                         |
| dimension                          | one of the seven things your score is built from               |
| perf / performance cases           | the test cases that check how fast your solution is            |
| refactor challenge                 | the improvement task you get after your first working solution |

## 4. How it works, step by step

The five phases run in a fixed order. You cannot go back to a finished phase.

| # | Phase             | What you do                                                                                                       |
|---|-------------------|-------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Understand</b> | Answer questions about the problem — what it is asking, what the edge cases are — before you see the editor       |
| 2 | <b>Plan</b>       | Lay out your approach in steps, and name the <b>data structures</b> your approach will need, from a provided list |

| # | Phase            | What you do                                                                                |
|---|------------------|--------------------------------------------------------------------------------------------|
| 3 | <b>Implement</b> | Write the actual solution and run it against the test cases                                |
| 4 | <b>Refactor</b>  | Improve your working solution against a challenge chosen for you (see below)               |
| 5 | <b>Explain</b>   | State the time and space complexity of your final solution and justify your design choices |

**The refactor challenge adapts to what you wrote.** Once your implementation is in, the platform measures how fast and how memory-efficient it actually is:

- **If your solution was already optimal** on both speed and memory and passed everything, the refactor phase is **skipped** and you score full marks on adaptability. Being asked to "make the optimal solution faster" is a question with no right answer, so you are not asked it.
- **If one dimension is lagging**, you get a challenge targeting exactly that — and you are scored on how much of the gap you closed against the target you were shown, never against a hidden bar.
- **Otherwise** you get the question's standard improvement challenge.

**Languages supported:** Python, JavaScript, Java, C++, C, Go and Rust.

## 5. Rules, limits and timings

| Rule                  | Value                                                               |
|-----------------------|---------------------------------------------------------------------|
| Attempts per question | <b>one per day</b>                                                  |
| Phase order           | Fixed; you cannot return to a finished phase                        |
| Time limit            | Each phase has its own limit, plus a <b>10-minute</b> overall grace |
| Code size limit       | 64 KB per submission                                                |

- **One attempt per question per day.** Trying again the same day gives you this exact message: "*You have already attempted this question today. Try again tomorrow.*" The day is counted from when the attempt **finished**, so starting just before midnight does not get you two attempts.
- Your work is saved as you go. If you close the tab and come back within the time limit, you resume where you were.
- If the overall time limit passes, the session expires and is scored on what you completed.

## 6. How scoring works

Your score is built from **seven dimensions**, weighted:

| Dimension           | Weight | What it measures                                                                                           | Which phase |
|---------------------|--------|------------------------------------------------------------------------------------------------------------|-------------|
| <b>Tests</b>        | 30%    | Whether your solution passes, weighted by category: basic cases 50%, edge cases 30%, performance cases 20% | Implement   |
| <b>Adaptability</b> | 18%    | Whether your refactor kept the tests passing and actually improved things                                  | Refactor    |

| Dimension              | Weight | What it measures                                                                                              | Which phase       |
|------------------------|--------|---------------------------------------------------------------------------------------------------------------|-------------------|
| <b>Decomposition</b>   | 12%    | How well you understood the problem and ordered your plan, including the edge cases you identified            | Understand + Plan |
| <b>Code structure</b>  | 12%    | The structural quality of the code you wrote                                                                  | Implement         |
| <b>Explanation</b>     | 12%    | Whether the complexity you claim matches what your code actually does, and whether your design claims hold up | Explain           |
| <b>Process</b>         | 10%    | How you worked — measured from your activity during the attempt                                               | All               |
| <b>Data structures</b> | 6%     | Whether the structures you named in the plan match what the problem needs                                     | Plan              |

Some principles worth knowing:

- **A refactor that breaks your tests is capped**, however much faster it got. Correctness first.
- **Claiming the wrong complexity costs you.** Your claim is checked against how your code actually performed. Being one step off scores partial credit; being far off scores none.
- **Naming every possible data structure does not help.** The data-structures score penalises both over-guessing and naming nothing.
- **If something could not be measured, it is not counted against you.** Where the platform cannot analyse your code or measure a performance ladder, that component is dropped and the rest is re-weighted — you are never given a zero for our inability to measure.
- **Scoring is deterministic.** The same work always produces the same score. There is no AI judgement in your Code Byte score, which is why a disputed score can be re-checked exactly.

## 7. What you get afterwards

---

- **Your composite score** and each of the seven dimension scores.
- **Per-phase breakdown** of what you did.
- **On a completed attempt**, the model solution: the canonical code in your language and a written explanation of why that approach is the best one.

The model solution only appears once you have **completed** the attempt. It is not shown mid-session or on an expired attempt.

## 8. When things go wrong

---

### "I passed all the tests but my score is not high"

Tests are 30% of the score. The other 70% covers whether you understood the problem, planned it, structured the code well, improved it, and explained it correctly. A passing solution with a wrong complexity claim and no plan scores much lower than the same code with both.

### **"I cannot start the question again today"**

One attempt per question per day. Try a different question, or come back tomorrow.

### **"My session expired"**

Each phase is timed and there is an overall limit with a 10-minute grace. Once that passes the attempt is closed and scored on what you finished.

### **"I cannot go back to a previous phase"**

That is intentional. The phases measure separate things, and going back to change your plan after seeing the tests would make the plan score meaningless.

### **"I did not get a refactor challenge"**

If your implementation was already optimal on both speed and memory and passed everything, the refactor phase is skipped and you get full marks for adaptability. This is a good outcome.

### **"My refactor was faster but my score dropped"**

Check whether it still passes the tests. A refactor that breaks the suite is capped regardless of speed.

### **"The complexity I claimed was right but scored badly"**

Your claim is checked against your own solution's measured performance. If the code did not actually behave the way you claimed — a hidden inner loop, an expensive library call — the measurement wins.

### **"My code will not run"**

Make sure the language you selected matches the code you wrote, and that your solution reads input and returns output in the shape the problem specifies.

## **9. Common questions**

---

**Which languages can I use?** Python, JavaScript, Java, C++, C, Go and Rust.

**Can I use my own editor and paste the code in?** You can write code however you like, but your activity during the attempt contributes to the process score, and proctoring rules may apply — check the rules screen before you start.

**How many attempts do I get?** One per question per day. Different questions are separate.

**Is the score AI-generated?** No. Code Byte scoring is deterministic and rule-based, which is why the same work always gets the same score and a disputed score can be rechecked exactly.

**Can I see the correct answer?** Yes, once you complete the attempt: the model solution in your language plus a written explanation of the approach. Not before, and not on an expired attempt.

**What is the difference between Code Byte and coding practice?** Coding practice is write-code-and-run-the-tests. Code Byte scores your whole problem-solving process across five phases.

**Does my Code Byte score affect my profile or XP?** Completing coding work earns XP. See [22-xp-levels-and-rewards.md](#).

## 10. What this does not do

---

- It does not let you return to a finished phase.
- It does not let you retry the same question twice in a day.
- It does not use AI to judge your code.
- It does not show the model solution before you finish.
- It does not score you for things it could not measure.

## 11. When to contact support

---

- A completed attempt that produced no score.
- A session that expired well before its displayed time limit.
- A dimension scored 0 where you believe you submitted work for that phase.